faithfulness, we first pre-compute mean activations for each model component (in each position) across *all* arithmetic prompts. We then intervene on the evaluation prompts by replacing non-circuit component activations with their means. To quantify performance, we measure $\mathrm{NL}(\mathbf{c})$, the logit of the correct answer token normalized by the maximal logit, as a proxy for accuracy, when mean-ablating all components not in the circuit $\mathbf{c}$. The circuit's faithfulness is calculated as:

$$\mathrm{F}(\mathbf{c}) = \frac{\mathrm{NL}(\mathbf{c}) - \mathrm{NL}(\emptyset)}{\mathrm{NL}(\mathbf{M}) - \mathrm{NL}(\emptyset)} \tag{2}$$

where $\mathbf{M}$ is the entire model and $\mathrm{NL}(\mathbf{M})$ is the normalized correct-answer logit when no component is ablated (always 1.0 for correctly completed prompts). $\mathrm{NL}(\emptyset)$ is the normalized correct answer logit when all components are mean-ablated. This formula normalizes faithfulness to a $[0.0, 1.0]$ range.

The circuit achieves a high faithfulness of 0.96 on average across the four arithmetic operators; i.e., the circuit accounts for 96% of the entire model's performance. We can therefore conclude that the components identified in this section comprise the arithmetic circuit, and explain most of the model's accuracy for arithmetic prompts. See Appendix A for results across various circuit sizes, and a discussion of Pareto-optimality with respect to faithfulness and size.

2.2 Identifying answer-promoting components

To understand the mechanism implemented by the circuit to promote the correct answer, we first search for the specific circuit components that increase the probability of the correct answer. For this, we employ linear probing (Belinkov, 2022). For each layer l and sequence position p, we train a linear classifier $f_{l,p} : \mathbb{R}^d \to \mathbb{R}^{1000}$ (where d is the dimension of each layer's output representation) using a training set of correctly completed arithmetic prompts. We pass these prompts through the model and calculate the output representation $h^{l,p} \in \mathbb{R}^d$ at layer l and position p for each prompt. The classifier $f_{l,p}$ receives $h^{l,p}$ as input, and outputs a probability distribution over the $1,000$ possible arithmetic answers. The classifier $f_{l,p}$ is evaluated on a separate test set of correctly completed prompts, showing to what extent the correct answer can be extracted from the output representation at layer l and position p.

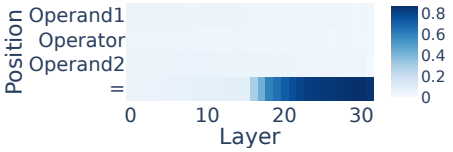


Figure 3: **Answer token probe accuracy.** The linear probes are successful in extracting the correct answer from the final position, starting at layer 16.

We find that the answer can only be extracted with high accuracy from the final position (Figure 3), after the token representation is processed by the later layers of the model, starting from layer $l = 16$. Given that the arithmetic circuit contains only MLPs in these layers (Figure 2), this suggests that these MLPs in layers $[16, 32]$ are the components that write the correct answer into the representation at the last position. The following section zooms into these middle- and late-layer MLPs, and presents evidence for the role they play in generating the correct answer— specifically, in how they promote the correct answer token through a combination of many independent arithmetic heuristics.

3 MLP neurons implement arithmetic heuristics

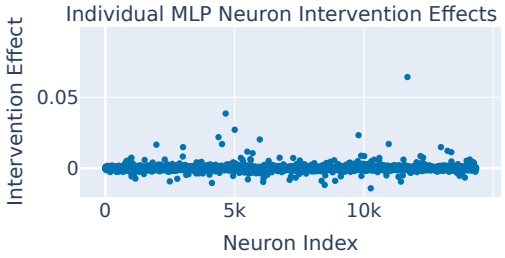
3.1 Decomposing circuit MLPs to individual neurons

Having shown that the model generates the arithmetic answer in middle- and late-layer MLPs at the final position $p = 4$, we zoom in on these MLPs and their calculations at this position to investigate the implemented mechanism. The MLP at layer l can be described by the following equation:

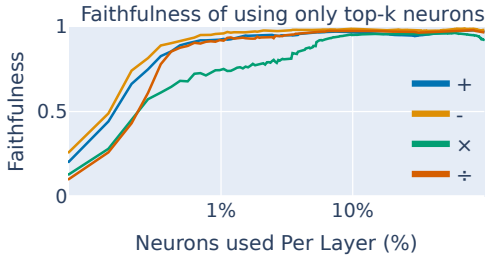
$$\mathbf{h}^l_{out} = \mathrm{MLP}_{in}\left(\mathbf{h}^l_{in}\right) \cdot \mathbf{W}^l_{out} = \mathbf{h}^l_{post} \cdot \mathbf{W}^l_{out} = \sum_{n=0}^{d_{mlp}} \mathbf{h}^{l,n}_{post} \mathbf{v}^{l,n}_{out} \tag{3}$$

where $\mathbf{h}^l_{in}, \mathbf{h}^l_{out} \in \mathbb{R}^d$ are the input and output representations of the MLP at layer l, respectively. $\mathbf{h}^l_{post} \in \mathbb{R}^{d_{mlp}}$ is the output of the up-projection of the MLP, [2] where we define the nth value $\mathbf{h}^{l,n}_{post} \in$

[2]The up-projection MLP_{in} is implemented differently in each LLM we analyze. See Appendix C.



(a) We intervene on each neuron in MLP layer $l = 17$. Few neurons in this and other middle- and late-layer MLPs have a high effect on arithmetic prompts.



(b) We measure the faithfulness when including only a fraction of high-effect neurons in the circuit. This circuit achieves high faithfulness.

Figure 4: Analyzing effect of individual circuit MLP neurons. Our results demonstrate that a small amount of neurons is required to correctly predict the result.

$\mathbb{R}$ as a neuron. $\mathbf{W}^l_{out} \in \mathbb{R}^{d_{mlp} \times d}$ is the output projection matrix, and $\mathbf{v}^{l,n}_{out}$ is its nth row vector. Biases are omitted. By expressing the output representation $\mathbf{h}^l_{out}$ as a linear combination of row vectors $\mathbf{v}^{l,n}_{out}$ and their corresponding neuron activations $\mathbf{h}^{l,n}_{post}$, we can identify the neurons that most affect the completion of arithmetic prompts.

To measure the effect of each neuron $\mathbf{h}^{l,n}_{post}$, we perform activation patching experiments on individual neurons (as described in Section 2.1), and measure the average effect across prompts. We observe that very few neurons have a high effect; an example for layer $l = 17$ is shown in Figure 4a. Additionally, we notice the neurons with the highest effect are different between operators. In fact, roughly 45% of the important neurons for each operator are unique (Appendix D). Thus, when analyzing the circuit at the neuron level, we analyze it as 4 separate circuits—one for each arithmetic operator. We hypothesize that for each operator, the highest-effect neurons are sufficient to explain most of the model's arithmetic behavior. To verify this, we measure the faithfulness of the arithmetic circuit when mean ablating non-circuit components and lower effect MLP neurons in middle- and late-layer MLPs. The results (Figure 4b) confirm that **only 200 neurons (roughly 1.5%) per layer are needed to achieve high faithfulness and correctly compute arithmetic prompts**.

3.2 MLP NEURONS ACT AS MEMORIZED HEURISTICS

To understand how the top-200 middle- and late-layer important MLP neurons contribute to the generation of correct answers, we view them as key-value memories (Geva et al., 2021).[3] In this view, the input to each MLP layer $\mathbf{h}^l_{in}$ is multiplied by a *key* (a row vector in the MLP input weight matrix) to generate a neuron activation $\mathbf{h}^{l,n}_{post}$ that determines how strongly does a *value* (a row vector $\mathbf{v}^{l,n}_{out}$ in the MLP output weight matrix) gets written to the MLP output (Equation (3)). Geva et al. (2021) demonstrated that keys correspond to specific topics or n-grams, triggering high neuron activations when these are given as input, and their corresponding values represent tokens that can serve as appropriate completions for these topics or n-grams. Building on this insight, we hypothesize that (i) in arithmetic contexts, keys correspond to numerical patterns, e.g., a neuron might activate strongly when both operands in an arithmetic operation are odd numbers; and (ii) the associated value vectors encode numerical tokens that represent plausible answers to the key patterns.

To test the first hypothesis, we investigate the activation pattern of the top-200 neurons in each layer. For each neuron l, n, we plot the activations $\mathbf{h}^{l,n}_{post}$ (at position $p = 4$) as a function of operand values, separately for each operator. We find that many neurons in the arithmetic circuit exhibit distinct, human-identifiable patterns. For instance, in "$226 - 68 =$", neuron $\mathbf{h}^{24,12439}_{post}$ shows high activation values for subtraction prompts with results between 150 and 180 (Figure 1). Additional examples are provided in Appendix J.

To verify the second hypothesis, we check whether the tokens embedded in the value vectors of the top neurons relate to their activation patterns. Using the Logit Lens (nostalgebraist, 2020), a method of projecting a vector $\mathbf{v} \in \mathbb{R}^d$ onto a probability distribution over the vocabulary space $\mathbb{P}^{d_{vocab}}$, we project each value vector $\mathbf{v}^{l,n}_{out}$ to find the numerical tokens whose logits are highest. This reveals

[3]Not to be confused with the attention heads' keys and values.

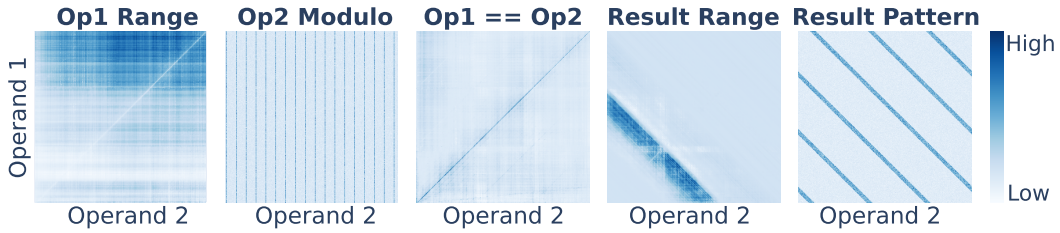


Figure 5: **Heuristic pattern examples.** Each heatmap is the activation pattern of an example neuron, implementing a specific **heuristic type**. Within the heatmap, each pixel at location (op_1, op_2) represents the activation strength of the neuron under the addition prompt "$op_1 + op_2 =$".

two distinct patterns: First, in some neurons, the activation pattern depends on both operands *and* the value vector encodes the expected result of the arithmetic calculation (Figure 1b,c). We term such neurons *direct heuristics*. Second, in neurons where the activation pattern depends on a single operand, the value vectors often encode features for downstream processing, rather than the result tokens directly (Figure 1a). We term such neurons *indirect heuristics*. Next, we demonstrate how these heuristic neurons combine to produce correct arithmetic answers.

4 Arithmetic prompts are answered with a bag of heuristics

Observing the example prompt, "$226 - 68 =$", we have shown that it satisfies the pattern of several heuristic neurons, where each such neuron slightly increases the logit of the result token, $r = 158$ (Figure 1). These small increases combine to promote the correct token as the final answer. We hypothesize that a combination of independent heuristics—termed a *bag of heuristics*—emerges across arithmetic prompts, comprising the mechanism used by the model to produce correct answers.

4.1 Classifying neurons to heuristic types

To present evidence for the causal effect of the bag of heuristics on generating correct answers, we first systematically classify neurons into heuristic types. Through manual observation of key activation patterns, we identify several categories of human-identifiable heuristics, exemplified in Figure 5, and further detailed in Appendix E. To determine if a neuron n at layer l implements a specific heuristic, we examine the intersection between the prompts that activate the neuron and the prompts expected to be activated for this heuristic. A visual example of this procedure is shown in Figure 6. An automated algorithm of this approach is described in Appendix F.

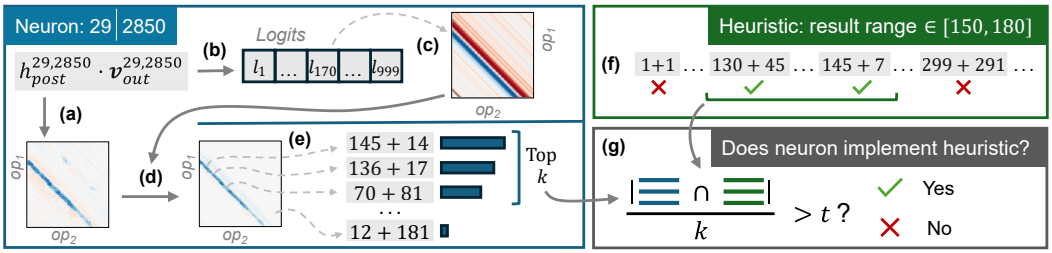


Figure 6: **Neuron to heuristic matching example.** **(a)** Measure the value of $h_{post}^{29,2850}$ for each operand pair (op_1, op_2), using the chosen operator (addition). **(b)** Calculate the logits of numerical tokens embedded in $v_{out}^{29,2850}$, using Logit Lens (nostalgebraist, 2020). **(c)** Convert the logits vector to a 2D pattern, where the cell in index (op_1, op_2) is the logit of the result token of applying the operator to (op_1, op_2) (i.e. $op_1 + op_2$). **(d)** Multiply both patterns element-wise, to get the effective logit contribution of the neuron to the correct answer token for each prompt. **(e)** Extract the prompts that activate the neuron the most from the activation pattern. **(f)** Create a list of prompts associated with the tested heuristic. **(g)** Measure the intersection between the two prompt lists. If this intersection is larger than a threshold (we use $t = 0.6$), the neuron is said to implement the heuristic.

We apply this algorithm to each pair of important MLP neuron n in layer l and heuristic H. Through this method, **we classify as arithmetic heuristics 91% of the 3,200 top neurons** for each operator